



Goblin++

第一天入门教程

简体中文译本 / 2026-10-03

Malin Hess

原教程版本：0.1.0-alpha.12。配套参考文档快照：0.1.0-alpha.21。

这是一份历史教程的完整翻译，不是将旧版内容冒充新版语言规范。安装包和编辑器版本请以本中文包的 START_HERE.md 为准。源码代码块保持原样；PDF 中过长代码行可能仅为排版而折行，复制运行时请使用配套 Markdown 或原始示例。

简体中文译本；对应 Goblin++ 0.1.0-alpha.21 文档快照。命令、标识符及代码示例保持原样。

译者说明--历史版本与当前版本：本文是原始 alpha.12 首日教程的完整简体中文译本，保留其版本号、日期、示例、限制和历史安装文件名；它不是 alpha.21 最新功能的完整教程。文中的 VS Code 0.1.7 等版本是历史版本，不是最新版。当前 alpha.21 的配套参考文档说明了后续变化：alpha.17 起加入电流这一第六个基本量纲；alpha.19 起加入本地模块、CSV/TSV 读取及原生 FITS / 文件输出支持，见 原生 I/O 指南；alpha.20 起保护常量名称并即时捕获模板，见 受保护的变量；alpha.21 使用可往返还原的默认数值文本，见 数值文本。当前安装及下载请参阅 简体中文 README，并以实际 CLI 的能力检查为准。不要把下文 alpha.12 的历史限制当作 alpha.21 的现有限制。

Goblin++

Alpha.12 首日教程

安装。编写。运行。验证。

基于 Rust 构建的实用科学编程语言

Malin Hess

版本 0.1.0-alpha.12 | 2026 年 9 月 24 日

关于本教程

本指南旨在帮助新用户从下载的 Goblin++ 归档包出发，在第一天就写出有用、可验证的科学程序。它教授的是 alpha.12 中已经存在的语言功能。计划中的功能明确标为未来工作，不会作为可用语法介绍。

最简洁而有用的定义 Goblin++ 是证据优先的科学编程语言。它结合普通编程功能、具有量纲意识的物理量、原生 FITS 访问、受控文件输出，以及保存实际执行内容和产出结果的保管链 workflow。

适用读者

- 希望编写可读、带单位且内置证据处理程序的科学家和学生。
- 希望简明学习变量、条件、循环、数组、函数和文件的新程序员。
- 评估 Goblin++ 是否适合作为可审计科学 workflow 语言的有经验程序员。
- 处理 FITS 图像或二进制表，希望显式访问数据并关联哈希的研究人员。

如何使用本指南

1. 完成第一部分，安装命令和编辑器支持。
2. 亲手输入第二部分的程序，而不只是阅读。
3. 准备好使用 FITS 输入和生成产物后，再使用第三部分。
4. 将结果当作证据或修订冻结工作之前，先使用第四部分。
5. 编写自己的程序时，将参考页放在手边。

译者说明：原教程上述部分编号与实际目录并不完全一致。按本书实际标题，安装在第二部分，首个程序在第三部分，科学数据在第五部分，证据与修订在第六部分。此处保留原文编号，避免无标记改写历史内容。

Alpha 状态 Alpha.12 经过测试且有实际用途，但这不构成语言稳定性的宣称。回执模式迁移、作者身份认证、更广泛的 FITS 语义、跨平台发布重现、编译执行的 FITS / 输出调用，以及模块仍是未完成工作。

许可与署名

版权所有 © 2026 Malin Hess。原始教程文字和图解依据 CC BY 4.0 提供。代码示例采用 MIT 许可。Goblin++ 标志是独立的项目品牌资产。依赖 crate 保留各自许可。确切范围见仓库中的许可文件。

目录

关于本教程	3
适用读者	3
如何使用本指南	3
许可与署名	3
目录	4
第一部分--认识 Goblin++	6
1. Goblin++ 解决什么问题？	6
2. 简短的发展历史	6
第二部分--安装与熟悉环境	7
3. 安装 macOS arm64 本地包	7
4. 从源码构建	7
5. 安装 VS Code 扩展	7
6. 创建工作区	7
第三部分--你的第一个程序	9
7. 最小的实用程序	9
8. 名称、赋值、注释和字符串	9
9. 值与类型	10
10. 单位与量纲检查	10
11. 注册常量	11
12. 运算符与优先级	11
第四部分--日常语言功能	12
13. 打印与格式化	12
14. GO_PARANOID 与 seal	12
15. 条件	12
16. 循环	13
17. 数组和切片	13
18. 文本与 g_strings	13
19. 输入与程序参数	14
20. G funk：用户自定义函数	14
21. 解释执行与编译执行	14
第五部分--科学数据与产物	16
22. 分析前先检查 FITS 文件	16

23. 读取 FITS 图像	16
24. 读取 FITS 二进制表	16
25. 显式筛选与加权统计	17
26. 写入文本、CSV、TSV 和 JSON	17
27. 创建确定性图表	18
第六部分--证据、保管链与修订	19
28. 运行保存什么	19
29. 冻结、运行、验证	19
30. 创建修订并检查谱系	19
31. 比较运行并审计项目状态	19
32. 内联 Rust：强大，且有意设置门槛	20
第七部分--首日引导练习	21
33. 练习 A：接受比例报告	21
34. 练习 B：检查随附 FITS 测试样本	21
35. 练习 C：编译日常程序	22
第八部分--故障排查与科学习惯	23
36. 常见错误	23
37. 保持语义的调试顺序	23
38. 科学审查清单	23
第九部分--快速参考	25
39. 语言关键字与指令	25
40. 日常内置函数	26
41. FITS 内置函数	26
42. 输出内置函数	27
43. CLI 命令卡	27
44. 当前 alpha.12 限制	27
45. 术语表	28
46. 接下来做什么	28

第一部分--认识 Goblin++

1. Goblin++ 解决什么问题？

科学工作常常从一个小计算开始，却最终演变为相互交织的脚本、复制的数据、未标明的图表，以及难以重建的结果。Goblin++ 将执行和证据视为同一个工作流。普通运行会保存起始源码、stdout、stderr、回执、相关导入数据证据、生成文件，以及显式封存值。

这不会让科学结论变成真理。它使计算主张可以接受检查：哪些源码字节实际运行、访问了哪些输入、由哪个引擎执行、创建了哪些输出，以及这些保存的字节是否仍匹配其哈希。

目标	Goblin++ 的应对方式
可读的日常代码	变量、文本、数组、函数、分支、循环、输入和命令行参数。
科学正确性辅助	物理量纲、注册常量、显式 FITS 函数、拒绝 NaN / 无穷大，以及有上限的操作。
可重现证据	唯一运行目录、SHA-256 回执、验证、冻结、修订谱系、语义差异比较和保管链账本。
原生性能路径	默认使用 Rust 解释器；对受支持的语言功能显式进行原生编译。

证据不等于真理 完全通过验证的运行仍可能编码不良模型、错误选择或错误假设。Goblin++ 帮助保存计算过程。科学验证仍需要独立推理、数据检查和领域审查。

2. 简短的发展历史

Goblin++ 起始于关注可审计科学计算的 Python 参考实现。早期 0.0.x 工作建立了规范化程序哈希、运行回执、验证、语义差异比较、精确字节冻结、修订谱系、校验和保管链账本、规范性语言资料、教程，以及 VS Code 编辑支持。

0.1.0 alpha 系列将引擎迁移到 Rust，同时保留证据优先设计并新增日常语言功能。

阶段	里程碑
Python 0.0.x	审计回执、verify/diff、冻结及修订谱系、保管链工具、语言参考、教程、编辑器支持。
Rust alpha.1-alpha.3	原生解释器 / 编译器基础、精确哈希授权的内联 Rust、原生 FITS 访问、经审计的文件和图表。
alpha.4-alpha.6	区间、while 循环、布尔比较、if/else、switch/case、可选日常模式和严格审计模式。
alpha.7-alpha.8	交互输入、argc/argv、一维值复制数组和切片。
alpha.9-alpha.10	用户自定义 g_func 函数和实用字符串操作。
alpha.11	有大小限制的筛选 / 加权 FITS 表摘要，采用明确的选择语义。
alpha.12	and/or/not、break/continue、直接数组迭代、受检余数和 parse_integer。

在 Rust 移植关卡完成之前，Python 0.0.7 参考版本仍是规范性兼容锚点。Alpha.12 是如实描述进展的开发版本，并不宣称每个旧功能或计划功能都已经迁移。

第二部分--安装与熟悉环境

3. 安装 macOS arm64 本地包

经审查的本地包包含 macOS arm64 可执行文件和完整源码。在 Finder 或命令行中解压，在解压后的目录中打开终端，然后运行：

终端

```
./install.sh --prefix "$HOME/.local"  
rehash  
goblin++ --version
```

预期输出包括 goblin++ 0.1.0-alpha.12。安装器将命令放在 ~/.local/bin/goblin++。

如果找不到命令 将 \$HOME/.local/bin 添加到 shell 的 PATH。对于 zsh，将 export PATH="\$HOME/.local/bin:\$PATH" 加入 ~/.zshrc，打开新终端，再次运行 goblin++ --version。

4. 从源码构建

源码构建要求 Rust 1.92 或更新版本。在仓库根目录运行：

终端

```
cargo test --locked  
./install.sh --prefix "$HOME/.local" --build-from-source  
goblin++ doctor .  
goblin++ --version
```

锁文件固定 crate 版本，Cargo 验证注册表校验和。本地构建成功本身不是跨平台发布声明；公开发布检查清单仍要求干净的 macOS 和 Linux 工作流程结果。

5. 安装 VS Code 扩展

1. 打开 Visual Studio Code。
2. 打开 Extensions，选择三点菜单，然后选择 Install from VSIX...
3. 选择 goblinpp-vscode-0.1.7.vsix，如有提示则重新加载。
4. 打开包含 .gbl 文件的文件夹。不要在不知道哪个项目拥有运行账本的情况下编辑孤立的科学文件。
5. 如果扩展找不到命令，将 Goblin++: Executable Path 设为 goblin++ 的绝对路径。

扩展提供文件识别、语法着色、补全、代码片段、悬浮帮助、文档符号、可选图标装饰，以及明确的检查、运行、编译运行、冻结、修订、验证、状态、谱系、账本审计和 doctor 命令。

权威边界 编辑器提示是辅助，不是语言权威。Rust CLI 决定语法是否有效、程序是否允许运行，以及证据是否通过验证。

6. 创建工作区

终端

```
mkdir my-goblin-programs  
cd my-goblin-programs  
goblin++ doctor .
```

Goblin++ 按需创建项目保管链状态。如果希望它们形成同一个项目历史，请将源码、运行目录和 .goblin 状态一起保留。对大型研究输入制定明确的存储和备份策略。

第三部分--你的第一个程序

7. 最小的实用程序

创建 acceptance_ratio.gbl :

acceptance_ratio.gbl

```
samples = 12
accepted = 9
fraction = accepted / samples

print("accepted fraction = {fraction:.3f}")
seal fraction
```

终端

```
goblin++ check acceptance_ratio.gbl
goblin++ acceptance_ratio.gbl
```

运行打印 accepted fraction = 0.750，并报告唯一的 RUN_DIR。check 是只读预览：解析文件但不创建证据，也不消耗输入。普通命令通过 Rust 解释器执行并保存证据。

阅读运行摘要

行	含义
RUN_STATUS=PASS	运行时完成执行，并通过其策略关卡。
EXECUTION_ENGINE=rust-interpreter	默认 Rust 求值器执行了 AST。
SOURCE_SHA256	精确 UTF-8 源码字节的摘要。
CANONICAL_SOURCE_SHA256	解析语义的摘要；记法别名可能共享它。
RECEIPT_CORE_SHA256	绑定核心回执字段的摘要。
RUN_DIR	包含源码、日志、回执、封存值、导入和输出的目录。
LEDGER_HEAD_SHA256	当前校验和账本链头；不是作者身份认证。

PASS 表示计算成功 它不表示样本计数收集正确，也不表示该比例回答了有效的科学问题。

8. 名称、赋值、注释和字符串

语言基础

```
# A comment continues to the end of the line.
project_name = "Night sky survey"
samples = 12
accepted = 9
complete = accepted == samples
print("{project_name}: complete = {complete}")
```

- 赋值使用 name = expression。没有 let 关键字。
- 名称使用字母、首字符之后的数字、下划线及部分科学符号。为便于跨环境协作，建议使用描述性 ASCII 名称。
- 字符串使用双引号，并支持 {name} 插值。

- 语句通常以换行结束。花括号界定控制流和函数体。
- Goblin++ 拒绝未知符号，而不是编造值。

9. 值与类型

类别	示例	重要规则
物理量	mass = 1 kg	存储为有限 f64 SI 值及五分量纲向量。
无量纲数	samples = 12	仍是物理量，但没有物理量纲。
布尔值	ready = true	条件要求 true/false；不根据数值推断真假。
文本	name = "Ada"	UTF-8 文本；操作区分大小写，不对 Unicode 进行规范化。
数组	values = [1, 2, 3]	一维、同类型、值复制，最多 100,000 个元素。

目前尚无独立的任意精度整数类型。parse_integer、区间参数、索引及余数限制为 $\pm(2^{53}-1)$ 内可精确表示的整数。

10. 单位与量纲检查

units.gbl

```

mass = 1 kg
distance = 2 km
time = 4 s
speed = distance / time
energy = mass * c^2

print("speed = {speed}")
print("energy = {energy}")

```

单位	含义	SI 换算因子
kg	千克	1
g	克	0.001 kg
m	米	1
km	千米	1000 m
s	秒	1
K	开尔文	1
mol	摩尔	1
J	焦耳	kg*m^2/s^2

Goblin++ 跟踪质量、长度、时间、温度和物质的量量纲。加法、减法及比较要求量纲一致。乘法、除法和整数幂推导新量纲。

拒绝：量纲不兼容

```
bad = 1 kg + 2 s
```

单位具有语义 单位不是装饰性文本。Goblin++ 在量纲错误变成貌似合理的数字之前就拒绝它。

11. 注册常量

别名	含义	状态 / 注册表
pi	数学常数 π	精确关系的浮点求值
c, speed_of_light	光速	精确值, SI 2019
h, planck_constant	普朗克常量	精确值, SI 2019
hbar	约化普朗克常量	导出关系, SI 2019
G, gravitational_constant	牛顿引力常量	测量值, CODATA 2022
k_B, boltzmann_constant	玻尔兹曼常量	精确值, SI 2019
N_A, avogadro_constant	阿伏伽德罗常量	精确值, SI 2019

运行回执记录实际使用的每个注册常量, 包括其规范化标识符、源码别名、SI 值、量纲、状态和注册表。未来的宇宙学常量注册表不属于 alpha.12。

12. 运算符与优先级

类别	运算符	说明
算术	+ - * / ^	文本 + 文本进行拼接; 物理量运算符具有量纲意识。
整数余数	%	仅允许安全无量纲整数; 除数不能为零。
比较	== != < <= > >=	物理量要求量纲一致。文本 / 布尔值仅支持相等比较。
布尔逻辑	not, and, or	仅允许布尔操作数; and/or 短路求值。
索引 / 切片	a[i], a[start:stop]	从零开始的索引; 左闭右开的复制切片。

绑定优先级由低到高为: or、and、not、比较、加法 / 减法、乘法 / 除法 / 余数、幂、一元正负号, 以及索引。如果科学条件可能被误读, 建议使用括号。

表达式

```
eligible = quality >= 3 and not warning
odd = sample_id % 2 != 0
energy = mass * c^2
```

第四部分--日常语言功能

13. 打印与格式化

输出

```
mass = 1.5 kg
fraction = 3 / 4
print("mass = {mass}")
print("fraction = {fraction:.3f}")
printf("fraction = %.3f", fraction)
```

字符串插值支持普通值字段及数值 .Nf 或 .Ne 格式。printf 支持有意保持小规模格式化子集。每次运行都保存标准输出并计算哈希。

14. GO_PARANOID 与 seal

严格审计模式的能量程序

```
GO_PARANOID

mass = 1 kg
energy = mass * c^2
print("Energy = {energy}")
seal energy
```

功能	它做什么	它不做什么
GO_PARANOID	执行后重新观察并保存源码字节，与起始字节比较；注册账本前自检查回执。	它不是持续监控、加密、恶意软件防护或操作系统沙箱。
seal name	将当前具名值快照为由 verify 检查的带类型、已计算哈希的产物。	print 或生成文件不要求它，它也不加密该值。

学习和普通脚本可使用日常模式。需要更严格的源码运行后证据策略时使用 GO_PARANOID。希望作为具名科学结果收集并验证的值，使用 seal。

15. 条件

branching.gbl

```
fraction = 9 / 12

if fraction >= 0.75 {
    verdict = "meets threshold"
} else if fraction >= 0.5 {
    verdict = "review"
} else {
    verdict = "below threshold"
}

switch verdict {
    case "meets threshold" { code = 1 }
    case "review" { code = 2 }
    default { code = 3 }
}

print("{verdict}; code = {code}")
```

仅执行选中的 if 分支。switch 对选择表达式求值一次，然后依次求值标签，直到第一个精确匹配。它不会贯穿执行。default 可省略、必须唯一且在最后。应谨慎使用精确浮点相等；对近似科学值使用容差条件。

16. 循环

循环

```
sum = 0
for i in range(1, 6) {
    sum = sum + i
}

for value in [1, 2, 3, 4, 5] {
    if value % 2 == 0 { continue }
    if value > 3 { break }
    print("odd value = {value}")
}

remaining = 3
while remaining > 0 {
    remaining = remaining - 1
}
```

- range(stop)、range(start, stop) 和 range(start, stop, step) 不包含终止值。
- 直接数组循环仅对一个独立数组值求值一次。
- break 退出最近一层循环；continue 开始它的下一次迭代。
- 所有循环体共享每次运行一百万次迭代的上限，包括嵌套循环。
- 上限捕获普通的意外无限循环；它不是完整资源沙箱。

17. 数组和切片

arrays.gbl

```
measurements = [1 kg, 2 kg, 3 kg, 4 kg]
selected = measurements[1:3]
selected[0] = 20 kg
extended = append(measurements, 5 kg)
count = len(measurements)

print("original = {measurements}")
print("slice = {selected}")
print("extended = {extended}")
```

数组元素必须全部是文本、全部是布尔值，或全部是相同量纲的物理量。不支持嵌套数组。索引是精确的非负无量纲整数。切片左闭右开且相互独立：修改 selected 不会改变 measurements。append 返回新副本。

18. 文本与 g_strings

字符串

```
name = str_trim(" Ada pi ")
label = "Researcher: " + name
characters = len(name)
found = str_contains(name, "pi")
changed = str_replace(name, "pi", "Lovelace")
parts = str_split("red,green,blue", ",")
colors = str_join(" / ", parts)
```

```

number = parse_number("2.5")
count = parse_integer("42")

print("{label}; characters = {characters}")
print("{colors}; number = {number}; count = {count}")

```

`len(text)` 计数 Unicode 标量值，而不是字节或字素簇。操作区分大小写，不规范化 Unicode。结果上限为 1,048,576 个 UTF-8 字节；分割 / 拼接最多支持 100,000 个部分。

转换是显式的 `input` 和 `argv` 返回文本。对有限无单位十进制文本使用 `parse_number`，对受检的十进制整数语法使用 `parse_integer`。两个函数都不解析单位。请在 `Goblin++` 表达式中写科学单位。

19. 输入与程序参数

交互

```

name = input("Please enter your name: ")
print("Hello, {name}!")

print("argc = {argc}")
program = argv(0)
first_argument = argv(1)

```

终端

```

goblin++ greeting.gbl
goblin++ program_args.gbl -- Ada

```

隐私边界提示、响应和命令行参数会未经脱敏地保存在通过哈希关联的交互证据中。除非明确希望保留它们且访问得到控制，否则不要输入密码、令牌、个人数据或保密研究值。

20. G funk：用户自定义函数

functions.gbl

```

g_func mean_of_two(a, b) {
    total = a + b
    return total / 2
}

mean_mass = mean_of_two(2 kg, 4 kg)
print("mean mass = {mean_mass}")
seal mean_mass

```

- 在顶层声明 `g_func`；调用可出现在声明之前。
- 参数复制到新局部环境。不会隐式捕获调用方变量。
- 每条实际到达的执行路径必须执行 `return expression`。
- 函数最多有 32 个参数；活动调用深度限制为 16。
- 将 `GO_PARANOID`、`seal`、嵌套声明和内联 `Rust` 保持在函数之外。

21. 解释执行与编译执行

终端

```

# Default: execute with the Rust interpreter
goblin++ experiment.gbl

# Compile, execute, and preserve compiler evidence in the run

```

```
goblin++ experiment.gbl --compile
```

```
# Produce a standalone native program without executing it
goblin++ compile experiment.gbl -o experiment
./experiment
```

编译是显式的。Goblin++ 生成可供审查的 Rust，不经 shell 调用 rustc。编译执行保存生成源码、原生二进制文件、原生结果清单、stdout、stderr 和哈希。

当前编译器边界 Alpha.12 拒绝对 FITS 调用和生成输出调用进行原生编译，包括隐藏在函数或不可达分支中的调用。通过默认 Rust 解释器运行这些程序。解释器自身就是原生 Rust。

第五部分--科学数据与产物

22. 分析前先检查 FITS 文件

终端

```
goblin++ fits-info catalogue.fits --quick
goblin++ fits-info catalogue.fits
goblin++ fits-info catalogue.fits --json
```

快速检查读取结构头，但有意跳过完整文件哈希，并标记为未计算哈希的初步探查。完整检查计算 SHA-256，但仍不创建运行回执。证据级导入发生在 .gbl 程序执行时。

索引	约定
HDU	从零开始：主 HDU 为 0，第一个扩展为 1。
表格行	从零开始。
重复列元素	从零开始。
图像轴	从一开始，与 FITS NAXIS 记法一致。

23. 读取 FITS 图像

fits_image.gbl

```
GO_PARANOID

file = "sample.fits"
hdus = fits_hdu_count(file)
width = fits_axis(file, 0, 1)
pixels = fits_count(file, 0)
first_pixel = fits_pixel(file, 0, 0)
image_mean = fits_mean(file, 0)

print("HDUs = {hdus}; pixels = {pixels}")
print("first pixel = {first_pixel}; mean = {image_mean}")
seal image_mean
```

支持的图像 BITPIX 值为 8、16、32、64、-32 和 -64。遵循 BSCALE、BZERO 和整数 BLANK。fits_mean 使用有限、非 BLANK 值。

24. 读取 FITS 二进制表

fits_table.gbl

```
file = "sample.fits"
table_name = fits_header(file, 1, "EXTNAME")
rows = fits_rows(file, 1)
columns = fits_columns(file, 1)
first_object = fits_column(file, 1, "OBJECT", 0)
valid_z = fits_column_valid_count(file, 1, "Z")
mean_z = fits_column_mean(file, 1, "Z")
minimum_z = fits_column_min(file, 1, "Z")
maximum_z = fits_column_max(file, 1, "Z")

print("table = {table_name}; rows = {rows}; columns = {columns}")
print("first object = {first_object}; valid Z = {valid_z}")
```



```
print("Z range = {minimum_z} to {maximum_z}; mean = {mean_z}")
```

直接访问空值单元格产生 G601。聚合统计跳过整数 TNULL 值和浮点 NaN，并使用补偿求和。列匹配不区分 ASCII 大小写。直接读取重复数值 / 逻辑列时要求元素索引。

25. 显式筛选与加权统计

概念布局--alpha.12 源码中的调用应保持在同一行

```
GO_PARANOID

stats = fits_select_stats(
    "sample.fits", 1,
    "Z", 0, 3,
    "Z", "QUALITY"
)

selected_rows = stats[0]
used_rows = stats[1]
weight_sum = stats[2]
weighted_mean_z = stats[3]

print("selected = {selected_rows}; used = {used_rows}")
print("weight sum = {weight_sum}; mean Z = {weighted_mean_z}")
seal stats
```

语法说明 Alpha.12 的语句通常在换行处结束。在可执行文件中，请将 fits_select_stats 调用保持在同一行，如 examples/fits_selection.gbl 所示。上面的展开布局只是为了便于阅读各参数组。

选择条件为 $\text{lower} \leq \text{selection} < \text{upper}$ 。值或权重缺失的选中行计入选中数量，但不用于计算。非缺失权重必须严格为正。结果为 [selected_rows, used_rows, weight_sum, mean]。不推断星表筛选条件、掩码、不确定度模型或宇宙学解释。

26. 写入文本、CSV、TSV 和 JSON

概念布局--alpha.12 源码中的调用应保持在同一行

```
write_text(
    "summary.txt",
    "Acceptance report",
    "fraction = {fraction:.3f}"
)

write_csv("summary.csv", 2,
    "metric", "value",
    "samples", samples,
    "fraction", fraction)

write_tsv("summary.tsv", 2,
    "metric", "value",
    "samples", samples)

write_json("summary.json",
    "samples", samples,
    "fraction", fraction)
```

生成文件只位于 RUN_DIR/outputs 下。名称必须是扁平普通文件名；绝对路径、目录、父目录遍历和重复声明会被拒绝。每个输出上限为 64 MiB。JSON 保留有量纲物理量的 SI 值和量纲向量，而不是悄悄丢弃单位。

27. 创建确定性图表

概念布局--可执行调用使用单行

```
plot_fits_histogram(  
    "redshift.png", file, 1, "Z",  
    40, 50000, "Sampled redshift distribution")  
  
plot_fits_scatter(  
    "z_error.svg", file, 1, "Z", "Z_ERR",  
    50000, "Redshift and reported error")
```

绘图使用确定性的等间隔行索引，跳过空值 / NaN，并将采样元数据和输入 FITS 哈希绑定到回执。支持 SVG 和无损 PNG。权威输出路径有意省略 JPEG。

图表是视图，不是选择规则 图标题和回执描述确定性采样。它们不会把采样点变成全总体结果，也不会施加巡天专用质量筛选。

第六部分--证据、保管链与修订

28. 运行保存什么

证据	用途
source.gbl / source metadata	精确起始程序字节和规范化语义。
stdout.log / stderr.log	用户可见的输出和失败。
receipt.json	结构化状态、哈希、执行详情、导入、产物和策略证据。
sealed_artifacts	由 seal 创建的带类型快照。
outputs	生成的文本、表格、JSON、SVG 和 PNG 文件。
data evidence	通过哈希关联的导入 FITS 字节和访问描述。
compiler evidence	编译执行的生成 Rust、二进制文件和原生结果清单。

验证重新计算已保存证据的哈希。它不重新运行科学程序，也不证明源码体现了正确假说。

29. 冻结、运行、验证

终端

```
goblin++ freeze experiment.gbl
goblin++ experiment.gbl
goblin++ verify runs/RUN_DIRECTORY
goblin++ status experiment.gbl
```

冻结固定精确源码字节、规范化程序、常量注册表和回执。后续任何源码字节变化都会在求值前被拒绝。如果编辑只改变记法而不改变规范化语义，Goblin++ 会报告该区别，但仍是协议违规，因为冻结的是精确字节。

冻结有意采用严格规则 不要通过覆盖历史让冻结后的编辑通过。请恢复字节，或创建显式子修订。

30. 创建修订并检查谱系

终端

```
goblin++ revise experiment.gbl experiment_R1.gbl --reason "correct the documented selection interval"

# Edit the child, then freeze and run it.
goblin++ freeze experiment_R1.gbl
goblin++ experiment_R1.gbl
goblin++ lineage experiment_R1.gbl
```

修订要求原因、拒绝覆盖、将源码复制到子版本，并写入引用冻结父版本的谱系回执。之后可有意编辑、冻结和运行子版本。

31. 比较运行并审计项目状态

终端

```
goblin++ diff runs/RUN_A runs/RUN_B
goblin++ audit-ledger .
goblin++ doctor .
goblin++ capabilities
```

```
goblin++ status experiment.gbl
goblin++ lineage experiment_R1.gbl
```

命令	它回答的问题
diff	源码字节、规范化程序、封存结果、stdout/stderr、环境或交互是否不同？
audit-ledger	本地校验和链是否能验证到其已记录的链头？
doctor	此项目能否运行、写入、编译，并信任其当前账本状态？
capabilities	这个确切二进制文件声称实现哪些功能和限制？
status	此源码是未冻结、已冻结且验证通过，还是不匹配？
lineage	哪条显式父 / 子修订链产生了此源码？

diff 分类为以下之一：IDENTICAL_RESULT、NOTATION_ONLY_CHANGE、NOTATION_ONLY_REVISION、NOTATION_ONLY_CHANGE_AFTER_FREEZE、RUNTIME_DIVERGENCE 或 SEMANTIC_OR_RESULT_CHANGE。应同时阅读逐字段比较和标签；标签是摘要，不能代替检查。

仅校验和的作者身份状态 alpha 账本检测哈希链篡改，但不认证事件创建者身份。它报告 CHECKSUM_ONLY。Ed25519 签名和中断的头检查点恢复仍待实现。

32. 内联 Rust：强大，且有意设置门槛

inline_rust.gbl

```
GO_PARANOID
x = 1

RUST_INLINE_BEGIN
println!("reviewed native operation");
RUST_INLINE_END

seal x
```

终端

```
goblin++ check inline_rust.gbl
goblin++ inline_rust.gbl --compile --allow-inline-rust FULL_64_CHARACTER_SHA256
```

解释器模式始终拒绝内联 Rust。编译执行模式要求每个已审查块的精确 SHA-256。改变一个字节就会使授权失效。生成的 Rust 和二进制文件会保存并计算哈希。

任意原生权限 获授权的内联 Rust 拥有运行它的用户的文件系统和网络权限。内容哈希准确记录批准了什么；它不是沙箱。仅在真正的操作系统沙箱或一次性环境中运行不受信任的块。

第七部分--首日引导练习

33. 练习 A：接受比例报告

目标：计算比例、分类、写一份小报告、保存结果，并验证运行。

概念源码--在 alpha.12 中将每个函数调用放在同一行

```
GO_PARANOID

samples = 12
accepted = 9
fraction = accepted / samples

if fraction >= 0.75 {
    verdict = "meets threshold"
} else {
    verdict = "review"
}

print("accepted fraction = {fraction:.3f}")
print("verdict = {verdict}")
write_text("acceptance.txt",
    "accepted fraction = {fraction:.3f}",
    "verdict = {verdict}")
write_json("acceptance.json",
    "samples", samples,
    "accepted", accepted,
    "fraction", fraction,
    "verdict", verdict)
seal fraction
seal verdict
```

1. 保存为 acceptance_report.gbl。
2. 运行 goblin++ check acceptance_report.gbl。
3. 运行 goblin++ acceptance_report.gbl，复制所报告的 RUN_DIR。
4. 打开 RUN_DIR/outputs，检查两个文件。
5. 运行 goblin++ verify RUN_DIR。
6. 将 9 改为 8，再次运行。使用 goblin++ diff RUN_A RUN_B 查看哪些证据改变了。

预期推理 第二次运行不应仅描述为“输出不同”。源码、规范化程序、stdout、具名封存值和生成文件都改变了。比较使你能确切指出哪些层发生变化。

34. 练习 B：检查随附 FITS 测试样本

终端

```
goblin++ fits-info examples/sample.fits --quick
goblin++ examples/fits_import.gbl
goblin++ verify examples/runs/RUN_DIRECTORY
```

1. 识别主图像和 CATALOG 二进制表扩展。
2. 运行示例前预测 HDU 数、图像像素数和表格行数。
3. 阅读 stdout.log，与终端输出比较。

4. 检查 receipt.json 中的导入数据摘要和访问描述。

5. 运行 fits_selection.gbl，解释选中行与使用行的区别，再解释均值。

随附的三行 FITS 文件是确定性的功能测试样本，不是科学 DESI 结果。其体积小，适合学习证据流及空值 / 权重行为。

35. 练习 C：编译日常程序

everyday_alpha12.gbl

```
values = [1, 2, 3, 4, 5, 6]
sum = 0

for value in values {
  if value % 2 != 0 { continue }
  if value > 4 { break }
  sum = sum + value
}

limit = parse_integer("10")
within_limit = sum <= limit and not sum == 0
print("sum = {sum}; within limit = {within_limit}")
seal sum
seal within_limit
```

终端

```
goblin++ everyday_alpha12.gbl
goblin++ everyday_alpha12.gbl --compile
goblin++ diff runs/INTERPRETED_RUN runs/COMPILED_RUN
```

两个引擎都应打印 sum = 6; within limit = true。运行比较应区分执行引擎，同时显示科学结果和封存值匹配。

第八部分--故障排查与科学习惯

36. 常见错误

错误代码 / 症状	可能原因	首先如何处理
command not found	~/local/bin 不在 PATH 中，或 shell 尚未刷新。	运行已安装的绝对路径、更新 PATH，并打开新 shell。
G002 parse failure	语法错误、缺少花括号、意外 token 或不支持的换行。	运行 check；阅读报告的位置；缩减为最小失败语句。
G101 UNKNOWN SYMBOL	名称拼错、缺少赋值、未知函数或不支持的功能。	检查拼写和作用域；不要只为消除错误而添加占位值。
G201 incompatible dimensions	不同量纲之间进行加法 / 比较。	检查单位及所需物理方程。
G202 numeric failure	除法 / 余数除以零、不安全整数、NaN / 无穷大或无效数值文本。	验证输入和定义域假设。
G203 type/control failure	值类别错误、无效索引、非布尔条件、循环上限或函数未返回。	阅读确切消息；验证值类型和实际到达的路径。
G405 self-check failure	严格审计模式证据无法保存或验证。	将运行视为失败；检查源码和证据存储。
G501 compile failure	不支持的编译调用、rustc 问题或生成源码错误。	对 FITS / 输出调用使用解释器；检查已保存的编译器 stderr。
G601 FITS failure	缺少关键字卡 / 单元格、不支持的结构、直接读取空值单元格或输入格式错误。	运行 fits-info；确认 HDU、列、行及受支持格式。

37. 保持语义的调试顺序

1. 执行前先运行 `goblin++ check file.gbl`。
2. 阅读完整错误，包括代码和所述边界。
3. 确认输入文件、HDU、行、单位和变量名，而不是猜测。
4. 缩减程序，同时保留失败表达式。
5. 冻结工作需要修改时，使用新源码或显式修订。
6. 也验证失败运行。已保存失败可以是有效的拒绝证据。
7. 修复之后比较新旧运行，知道哪个层发生变化。

不要通过抹除历史来调试 删除失败运行或覆盖冻结文件可能让文件夹看起来整洁，却摧毁理解失败所需的证据。保留有用内容；只丢弃明确的一次性实验。

38. 科学审查清单

- 选择列、筛选条件、权重或单位之前先明确科学问题。
- 计算聚合结果之前检查数据结构和缺失值行为。
- 将采样图标记为采样图；不要暗示全总体统计。

- 谨慎处理精确浮点相等；根据科学要求而不是便利性定义容差。
- 只封存有意义的输出，但通过运行系统保存所有生成产物。
- 验证报告中使用的每次运行，记录其 RUN_DIR 和回执摘要。
- 用修订原因解释科学意图，而不只是说明文件改变了。
- 利用已知值或独立实现对重要结果进行独立验证。
- 除非明确安排存储和访问控制，否则不要将敏感数据放入明文证据。
- 如实报告不支持的语义。拒绝操作比编造解释更安全。

第九部分--快速参考

39. 语言关键字与指令

词	用途
GO_PARANOID	顶层选择启用的源码运行后证据和回执自检查。
seal	将具名值快照为带类型产物。
g_func	声明顶层用户函数。
return	从 g_func 立即返回值。
for / in / range	区间或直接数组迭代。
while	布尔条件为真时重复。
break / continue	退出最近一层循环或进入其下一次迭代。
if / else	条件分支；else if 由这两个词组成。
switch / case / default	第一个精确匹配的 case，无贯穿执行。
and / or / not	短路布尔逻辑和取反。
true / false	布尔字面值。
argc	包括 argv(0) 的只读参数数量。
RUST_INLINE_BEGIN / END	通过精确哈希授权的任意 Rust 块边界。

40. 日常内置函数

调用	用途
<code>print(values...)</code>	将插值或渲染后的一行写到已计算哈希的 <code>stdout</code> 。
<code>printf(format, values...)</code>	有限制的数值 / 文本格式化输出。
<code>input([prompt])</code>	读取一行 UTF-8 文本并保存交互证据。
<code>argv(index)</code>	读取从零开始编号的文本程序参数。
<code>len(value)</code>	数组元素数或文本 Unicode 标量值数。
<code>append(array, value)</code>	返回增加一个元素后的独立数组副本。
<code>to_text(value)</code>	可读的物理量 / 布尔值 / 文本渲染；拒绝数组。
<code>parse_number(text)</code>	受检的有限无量纲十进制转换。
<code>parse_integer(text)</code>	受检的安全十进制整数转换。
<code>str_trim(text)</code>	移除首尾 Unicode 空白。
<code>str_contains(text, needle)</code>	区分大小写的子串检查。
<code>str_replace(text, old, new)</code>	替换不重叠匹配；old 不能为空。
<code>str_split(text, separator)</code>	分割为文本数组；保留空字段。
<code>str_join(separator, array)</code>	拼接一维文本数组。

41. FITS 内置函数

调用	结果
<code>fits_hdu_count(file)</code>	HDU 数量。
<code>fits_header(file[, hdu], key)</code>	头中的值。
<code>fits_axis(file[, hdu], axis)</code>	图像轴长度。
<code>fits_count(file[, hdu])</code>	图像元素数量。
<code>fits_pixel(file[, hdu], index)</code>	展平后的图像值。
<code>fits_mean(file[, hdu])</code>	有限、非 BLANK 图像值的均值。
<code>fits_rows(file, hdu)</code>	二进制表行数。
<code>fits_columns(file, hdu)</code>	二进制表列数。
<code>fits_column(file, hdu, name, row[, element])</code>	标量或重复单单元格值。
<code>fits_column_valid_count(file, hdu, name)</code>	非空数值元素数量。
<code>fits_column_mean/min/max(file, hdu, name)</code>	有效数值元素的补偿聚合。
<code>fits_select_stats(file, hdu, selection, lower, upper, value[, weight])</code>	[selected, used, weight sum, mean]。

42. 输出内置函数

调用	输出
<code>write_text(name, values...)</code>	.txt 或 .md，每个参数一行。
<code>write_csv(name, columns, cells...)</code>	RFC4180 风格 CSV 表。
<code>write_tsv(name, columns, cells...)</code>	制表符分隔表，清理控制分隔符。
<code>write_json(name, key, value, ...)</code>	具有唯一文本键的 JSON 对象。
<code>plot_fits_histogram(name, file, hdu, column, bins, max_rows, title)</code>	确定性采样的 SVG 或 PNG 直方图。
<code>plot_fits_scatter(name, file, hdu, x, y, max_rows, title)</code>	确定性采样的 SVG 或 PNG 散点图。

43. CLI 命令卡

命令	用途
<code>goblin++ file.gbl</code>	解释执行并保存运行。
<code>goblin++ file.gbl --compile</code>	编译、执行并保存编译器证据。
<code>goblin++ run file.gbl -- args</code>	带程序参数的显式运行形式。
<code>goblin++ check file.gbl</code>	只读语法和策略预览；不产生证据。
<code>goblin++ compile file.gbl -o program</code>	创建独立原生可执行文件。
<code>goblin++ freeze file.gbl</code>	固定精确源码和规范化语义。
<code>goblin++ revise parent child --reason "..."</code>	创建显式子修订谱系。
<code>goblin++ verify RUN_DIR</code>	重新计算哈希并验证已保存证据。
<code>goblin++ diff RUN_A RUN_B</code>	对运行差异进行分类。
<code>goblin++ status file.gbl</code>	显示源码的冻结和账本状态。
<code>goblin++ lineage file.gbl</code>	显示修订祖先关系。
<code>goblin++ audit-ledger .</code>	验证本地校验和链。
<code>goblin++ doctor .</code>	检查项目准备情况。
<code>goblin++ capabilities</code>	列出已实现功能和真实限制。
<code>goblin++ fits-info file.fits [--quick --json]</code>	检查 FITS 结构。

44. 当前 alpha.12 限制

- 数字是 f64 物理量；没有任意精度整数类型。
- 数组是一维且同类型的；不支持嵌套集合。
- 尚无 Goblin++ 源码模块 / 导入系统。
- FITS / 输出调用仅支持解释执行，尽管解释器是原生 Rust。

- 不支持 FITS ASCII 表、分块压缩、随机组、位 / 复数值、变长数组堆、WCS 解释，以及自动 TUNIT 转换。
- 图表是确定性采样，不是全总体统计。
- 尚未实现 FITS 写入和表格批量导出。
- 运行证据是明文，未加密。
- 保管链账本建立校验和链，但不认证作者身份。
- 内联 Rust 是任意原生代码，不受信任时需要真正的沙箱。
- 跨平台公开发布重现和依赖声明审查仍是发布关卡。

45. 术语表

术语	含义
规范化程序	解析得到的语义结构，用于区分语义和源码记法。
回执	绑定状态、源码、日志、产物、导入、执行和哈希的结构化记录。
封存	对具名值显式建立带类型快照。
冻结	严格固定精确源码字节、规范化程序和注册表状态。
修订	通过必填原因关联到冻结父版本的显式子源码。
修订谱系	显式修订的父 / 子历史。
保管链账本	仅追加的项目事件 SHA-256 链；alpha 不认证作者身份。
仅记法变化	源码字节不同，规范化程序相同。
运行目录	包含日志、回执、证据和产物的唯一保存执行记录。
证据优先	将执行与可检查的来源信息一同处理的设计原则。

46. 接下来做什么

1. 运行 examples/ 中的每个程序，从 acceptance_ratio.gbl 开始，到 output_demo.gbl 结束。
2. 为自己的领域编写小程序，使用一个物理量、一个分支、一个函数和一个封存结果。
3. 使用 FITS 时，先检查结构，再记录为什么所选的每个列、边界和权重具有科学合理性。
4. 练习冻结源码、创建有理由的修订，并比较两次运行。
5. 用于正式研究之前，阅读 docs/SECURITY.md、docs/PORTING_MATRIX.md 和公开发布检查清单。

Goblin 规则 不轻信。给一切计算哈希。不覆盖。然后记住：哈希保存发生了什么；数据和测试揭示科学想法能否经受检验。

THE GOBLIN IS SATISFIED.

Goblin++ 0.1.0-alpha.12 | 教程第 1 版